



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

Praca dyplomowa inżynierska

na kierunku Informatyka
na specjalności Inżynieria oprogramowania

Integracja komponentu renderującego obiekty 3D z internetowym serwisem aukcyjnym

Integration of a component rendering 3D objects with an online auction service

Konrad Tomasz Nowak

99640

Maciej Ołdakowski

99648

Patryk Kamil Nowacki

99638

Promotor Dr Marcin Barszcz

Lublin 2025

Spis treści

1	Wstęp	4
2	Cel i zakres pracy	5
2.1	Cel pracy	5
2.2	Zakres pracy	5
3	Analiza rynku	7
3.1	allegro.pl	7
3.2	amazon.pl	7
3.3	erli.pl	7
4	Użyte technologie i narzędzia	9
4.1	Komponent renderujący	9
4.1.1	Język C++	9
4.1.2	WebGPU	9
4.1.3	WebGPU-Cpp	9
4.1.4	WebAssembly	9
4.1.5	Emscripten	9
4.1.6	WGSL (WebGPU Shading Language)	10
4.1.7	CMake	10
4.1.8	Ninja	10
4.1.9	Biblioteki C++	10
4.2	System aukcyjny - Frontend	11
4.2.1	React	11
4.2.2	React Router DOM	11
4.2.3	Tailwind CSS	11
4.2.4	TW Elements	11
4.3	System aukcyjny - Backend	11
4.3.1	Node.js	11
4.3.2	Express.js	11
4.3.3	MongoDB	11
4.3.4	Mongoose	11
4.3.5	JWT (jsonwebtoken)	12
4.3.6	bcryptjs	12
4.3.7	CORS	12
4.3.8	dotenv	12
4.3.9	validator	12
5	Wprowadzenie do renderowania 3D	13

5.1	WebGPU	13
5.2	WebAssembly	16
5.3	Podsumowanie	16
6	Implementacja komponentu renderującego	17
6.1	Okno aplikacji	17
6.2	WebGPU	19
6.2.1	Adapter	19
6.2.2	Urządzenie	20
6.2.3	Zasoby tworzone przez urządzenie	21
6.3	Pętla główna	24
7	Implementacja wyglądu serwisu aukcyjnego	26
7.1	Koncepcja interfejsu użytkownika	26
7.2	Prezentacja poszczególnych widoków	26
7.2.1	Strona główna	26
7.2.2	Strona szczegółów aukcji	27
7.2.3	System nawigacji	27
7.2.4	Strony logowania i rejestracji	27
8	Implementacja Bazy Danych oraz Logiki Biznesowej	29
8.1	Model Aukcji (Auction)	29
8.2	Model Licytacji (Bid)	30
8.3	Model Użytkownika (User)	30
8.4	Strategia Walidacji	31
8.5	Relacje między modelami	31
8.6	Implementacja REST API	32
8.6.1	Middleware Autentykacji	32
8.6.2	Kluczowe endpointy API	32
8.6.3	Format odpowiedzi	32
8.6.4	Przepływ danych — Złożenie licytacji	33
9	Wymagania sprzętowe	33
9.1	Przeglądarki desktopowe	33
9.2	Przeglądarki mobilne	34
10	Integracja	35
10.1	Kompilacja do WebAssembly	35
10.1.1	Konfiguracja CMake dla Emscripten	35
10.1.2	Generowane pliki wyjściowe	35
10.2	Architektura integracji w React	36

10.2.1	Wzorzec Provider	36
10.2.2	Hooki dostępu do API	36
10.3	Mechanizm React Portals	37
10.3.1	Zasada działania	37
10.3.2	Zarządzanie slotami	38
10.4	Ładowanie modułu WebAssembly	38
10.4.1	Konfiguracja obiektu Module	38
10.4.2	Asynchroniczne wstrzykiwanie skryptu	39
10.5	Wirtualny system plików	39
10.5.1	Montowanie systemu plików	39
10.6	Komunikacja JavaScript-C++	40
10.6.1	Eksportowane funkcje	40
10.6.2	Przekazywanie elementu canvas	41
10.7	Komponent WebGPUCanvas	41

1 Wstęp

Z roku na rok rynek e-commerce dynamicznie się rozwija, przyciągając coraz większą liczbę użytkowników. Według badania „E-commerce w Polsce 2024” szacuje się, że w 2024 roku aż 75% internautów dokonuje zakupów w polskich sklepach internetowych, a 36% korzysta z zagranicznych platform e-commerce. Pandemia Covid-19 znacząco przyspieszyła ten trend - w 2020 roku wartość sprzedaży online w Polsce podwoiła się w porównaniu z rokiem poprzednim. Wraz z rozwojem rynku użytkownicy oczekują coraz bardziej innowacyjnych rozwiązań, które zwiększają ich zaufanie do zakupów online i poprawiają jakość doświadczeń zakupowych.

Jednym z kluczowych wyzwań w handlu internetowym, w szczególności w sektorze aukcji online, jest zapewnienie kupującym możliwości dokładnego zapoznania się ze stanem oferowanego przedmiotu. Tradycyjne platformy aukcyjne, takie jak [desa.pl](#), [allegro.pl](#) czy [the-saleroom.com](#) opierają się głównie na zdjęciach i opisach, które nie zawsze w pełni oddają rzeczywisty stan produktu. Brak możliwości szczegółowego obejrzenia przedmiotu może prowadzić do nieporozumień i obniżać zaufanie kupujących. Rozwiązaniem tego problemu może być zastosowanie nowoczesnych technologii wizualizacji, które umożliwiają interaktywne przedstawienie produktów w trójwymiarowej przestrzeni.

W pracy inżynierskiej opisane zostanie wykorzystanie technologii WebGPU, użytej do przedstawienia trójwymiarowych obiektów. Obecnie najpowszechniejszym stosowanym standardem do renderowania grafiki 3D w przeglądarkach jest WebGL, na którym opiera się zdecydowana większość istniejących rozwiązań – od prostych wizualizacji produktów w sklepach internetowych po zaawansowane aplikacje typu konfigurator 3D czy przeglądarki modeli w branżach takich jak motoryzacja, meblarstwo i sztuka. Na WebGL bazują popularne biblioteki i silniki, m.in. [Three.js](#), [Babylon.js](#), [PlayCanvas](#) czy [Potree](#), które znacznie upraszczają tworzenie złożonych scen 3D.

Aktualnie powstaje próba wprowadzenia nowego standardu WebGPU - następcy WebGL, który oferuje dostęp do nowoczesnych interfejsów programistycznych (Vulkan, Metal, Direct3D 12) pozwalających na komunikację z procesorami graficznymi, znacznie wyższą wydajność, lepszą kontrolę nad pamięcią GPU oraz możliwość wykorzystywania shaderów obliczeniowych (`compute shaders`). W grudniu 2025 roku WebGPU jest już stabilnie wspierany w przeglądarkach opartych o silnik Chromium (Chrome, Edge, Opera) oraz w Firefoxie - w sumie w 80.71% przeglądarek. Dzięki temu możliwe staje się osiągnięcie jakości i płynności renderowania dotychczas zarezerwowanej dla aplikacji natywnych, przy jednoczesnym zachowaniu pełnej kompatybilności z ekosystemem webowym. Wykorzystanie WebGPU otwiera nowe możliwości tworzenia fotorealistycznych i wysoce interaktywnych wizualizacji przedmiotów aukcyjnych bezpośrednio w przeglądarce, bez konieczności instalowania dodatkowego oprogramowania.

2 Cel i zakres pracy

2.1 Cel pracy

Celem pracy inżynierskiej jest zaprojektowanie i implementacja systemu informatycznego, składającego się z dwóch zintegrowanych części: komponentu renderującego modele trójwymiarowe oraz internetowego serwisu aukcyjnego. Praca zakłada przedstawienie implementacji obu elementów systemu oraz sposobu ich połączenia w jeden działający system informatyczny. Część odpowiedzialna za wizualizację modeli 3D będzie wykorzystywać zasoby procesora graficznego w celu zapewnienia płynnego i efektywnego renderowania. Przekompilowany do formatu WebAssembly kod komponentu graficznego zostanie poprawnie uruchomiony przez nowoczesne silniki przeglądarek, oferując wydajność porównywalną z aplikacją uruchamianą natywnie na komputerze. Realizacja części aukcyjnej obejmuje mechanizmy typowe dla serwisów aukcyjnych: publikowanie ofert i zarządzanie licytacjami z odliczaniem czasu do zakończenia, obsługę ofert w czasie rzeczywistym. Interakcja z trójwymiarowym modelem licytowanego przedmiotu będzie możliwa bezpośrednio z poziomu karty aukcji.

2.2 Zakres pracy

Zakres pracy obejmuje następujące kluczowe elementy:

1. Wprowadzenie do technologii WebGPU i WebAssembly.
 - Omówienie podstawowych koncepcji i architektury WebGPU.
 - Opis technologii WebAssembly i jej zastosowań w kontekście aplikacji webowych.
2. Projekt i implementacja komponentu renderującego 3D.
 - Opis projektowania i implementacji komponentu renderującego.
 - Wyjaśnienie procesu kompilacji kodu napisanego w C++ do WebAssembly.
 - Omówienie potencjalnych trudności.
3. Analiza rynku platform aukcyjnych online.
 - Przeprowadzenie analizy głównych platform aukcyjnych.
 - Ocena aktualnie dostępnych technologii wizualizacji.
 - Identyfikacja potrzeb użytkowników i potencjalnych obszarów poprawy.
4. Projekt i implementacja części klienckiej serwisu aukcyjnego.
 - Zaprojektowanie układu stron.

- Stworzenie komponentów frontendowych.
5. Projekt i implementacja części serwerowej serwisu aukcyjnego.
 - Implementacja bazy danych oraz logiki biznesowej.
 - Implementacja ścieżek API.
 6. Integracja z systemem domu aukcyjnego.
 - Opis sposobu włączenia komponentu renderującego do interfejsu użytkownika.
 7. Ewaluacja i perspektywy rozwoju.
 - Testowanie wydajności i testy jednostkowe.
 - Przyszłe zastosowania.
 8. Wnioski i podsumowanie.

3 Analiza rynku

Na rynku istnieje wiele serwisów internetowych, na których użytkownicy mogą licytować i kupować przedmioty w czasie rzeczywistym. Jak okazuje się, nawet na największych platformach, nie ma możliwości podglądu obiektu w trzech wymiarach. Witryny takie jak allegro.pl, amazon.pl czy Erli posiadają prostą funkcjonalność podglądu zdjęć, nie wyróżniając się innowacyjnym rozwiązaniem jakim jest rendering 3D licytowanych obiektów. Obrazy 2D zapewniają statyczny widok i nie oddają skomplikowanych szczegółów, kątów lub faktur. Prowadzi to do tego, że nie znamy rzeczywistego stanu przedmiotu, przez co potencjalny konsument, może zostać wprowadzony w błąd, podczas zakupu. Technologia ta pozwala nam na dokładną inspekcję wybranego przez nas przedmiotu, poprzez obracanie obiektem, możliwością przybliżania oraz oddalania się od obiektu i renderingu w bardzo wysokiej jakości.

3.1 allegro.pl

Jest to największy serwis aukcyjny w Polsce, założony w 1999 roku przez Surf Stop Shop. Ponad 33% konsumentów zaczyna właśnie wyszukiwanie produktów na wyżej wymienionym serwisie. Podgląd obiektów na tej stronie jest bardzo prosty, gdyż pozwala nam tylko na podgląd zdjęć, bez możliwości zaawansowanego podglądu licytowanego obiektu. Interfejs użytkownika jest prosty i bardzo intuicyjny, a funkcjonalność licytowania pozwala wielu użytkownikom licytować te same przedmioty bez żadnych problemów. Na tym portalu każdy użytkownik może zostać sprzedawcą i wystawić własne przedmioty, nie korzystając z żadnego pośrednika, co ułatwia kontakt pomiędzy konsumentem a sprzedawcą, dzięki czemu każdy problem dotyczący sprzedaży może zostać rozwiązany w prosty sposób.

3.2 amazon.pl

Amazon to jeden z największych serwisów e-commerce na całym świecie. Startował jako księgrania internetowa, której asortyment systematycznie powiększał się, aż do dzisiejszych rozmiarów. Jedyną funkcjonalnością podglądu kupowanego przedmiotu jest przybliżenie obrazka, który zapewnił nam sprzedający. Nie mamy możliwości podglądu realnego stanu licytowanego przedmiotu. Sam proces kupowania czy licytowania jest bardzo intuicyjny i szybki, przez co użytkownik nie ma problemów. Amazon ma bardzo dobrze rozbudowaną obsługę klienta, przez co wszystkie problemy są rozwiązywane szybko i bezproblemowo.

3.3 erli.pl

Założona w 2020 polska firma, która zajmuje się sektorem e-commerce. Zajmuje się sprzedażą przedmiotów z kategorii: meble, budowa i remont, ogród, oświetlenie, narzędzia i wyposażenie domu. Strona ma prosty i przejrzysty układ strony kupowanego przedmiotu, lecz podgląd kupowanego przedmiotu ogranicza się tylko i wyłącznie do powiększenia zdjęć

oferowanych przez sprzedawcę. Takie rozwiązanie może nie oddawać dobrze faktur i tekstur przedmiotu, przez co użytkownik może zostać wprowadzony w skonfundowanie podczas zakupu. Wyświetlane zdjęcia są na małej części strony, co utrudnia ogląd licytowanego przedmiotu dla użytkowników, gdyż muszą powiększać stronę, aby dobrze obejrzeć zdjęcia zawarte przez sprzedającego.

4 Użyte technologie i narzędzia

4.1 Komponent renderujący

4.1.1 Język C++

Wysokopoziomowy język programowania stworzony przez Bjarne Stroustrupa, jako dodatek do języka C. C++ jest szeroko używany w aplikacjach wymagających wysokiej wydajności, takich jak gry komputerowe, silniki gier komputerowych, aplikacje graficzne, aplikacje uczenia maszynowego i wiele innych, gdzie wydajność jest kluczowa. Perfekcyjnie wpasowują się on w potrzebę stworzenia graficznego komponentu renderującego obiekty 3D.

W przedstawionym projekcie wykorzystywany jest C++ 23 (ISO/IEC 14882:2024), czyli aktualny otwarty standard języka. Ostateczna wersja dokumentu to N4950. Zawiera ona w sobie wiele nowych funkcji, oraz całą zawartość wcześniejszych standardów.

4.1.2 WebGPU

Nowoczesny interfejs programowania aplikacji (API), który umożliwia wydajny dostęp do procesora graficznego (GPU) na różnych platformach. U podstawy działania wykorzystuje systemowe interfejsy: Vulkan, Metal, lub Direct3D 12. Zastępuje starszą technologię WebGL, jako nowy główny standard graficzny dla sieci.

WebGPU jest wspierane zarówno w Google Chrome, jak i Firefox. Mozilla używa własnej implementacji w języku Rust o nazwie wgpu. Google natomiast stworzyło Dawn, czyli implementację standardu WebGPU w Chromium za pomocą C++.

4.1.3 WebGPU-Cpp

Wrapper dla WebGPU napisany w C++, ponieważ obie implementacje udostępniają plik nagłówkowy w języku C z definicjami funkcji i struktur. Stworzony został przez użytkownika eliemichel i udostępniany jest na platformie Github.

4.1.4 WebAssembly

Otwarty standard, który definiuje przenośny format binarny i odpowiadający mu format tekstowy dla programów komputerowych. Głównym celem jest umożliwienie łatwiejszego tworzenia wielce wydajnych aplikacji w przeglądarkach na stronach internetowych. Jest niezależny od platformy oraz wspiera każdy język programowania. Kod wykonywany jest w wirtualnej maszynie stosowej.

4.1.5 Emscripten

Całkowicie otwarty kompilator, który umożliwia kompilację kodu C i C++, lub innego języka używającego LLVM, do formatu WebAssembly. Emcc, czyli frontend kompilatora

używa Clang, oraz LLVM. Pozwala on na bezproblemową konwersję praktycznie każdego projektu C i C++ do WebAssembly. Emscripten ma na koncie kilka udanych konwersji takich jak: Unreal Engine 4, Quake 3, czy Doom 3, wszystkie działające w przeglądarce.

4.1.6 WGSL (WebGPU Shading Language)

Wysokopoziomowy język shaderów dla WebGPU. Umożliwia on tworzenie shaderów, czyli programów wykonywanych na procesorze graficznym. Został stworzony przez W3C GPU for the Web Community Group, aby zapewnić nowoczesny, bezpieczny i przenośny sposób pisania shaderów dla WebGPU.

4.1.7 CMake

Narzędzie do zarządzania budowaniem kodu źródłowego. Zdejmuje z użytkownika konieczność pisania skomplikowanych plików potrzebnych systemom budowania. Jest szeroko używany w projektach C i C++.

4.1.8 Ninja

Lekki system budowania, skupiający się na szybkości. Jest zaprojektowany, aby pliki wejściowe były generowane przez inne wysokopoziomowe narzędzie. Używany do budowania Google Chrome, czy części systemu Android.

4.1.9 Biblioteki C++

- GLFW - Napisana w C wieloplatformowa biblioteka do tworzenia okien aplikacji używających OpenGL, OpenGL ES i Vulkan.
- GLM (OpenGL Mathematics) - Napisana w C++ biblioteka matematyczna przeznaczona dla oprogramowania graficznego opartego na specyfikacjach języka OpenGL Shading Language (GLSL). Biblioteka ta doskonale współpracuje z OpenGL, ale zapewnia kompatybilność z innymi bibliotekami i zestawami SDK innych producentów.
- stb_image - Biblioteka dla C/C++ umożliwiająca załadowywanie obrazów w różnych formatach, takich jak: JPG, PNG, TGA, BMP, PSD, GIF, HDR, PIC.
- tinyobjloader - Napisana w C++ biblioteka umożliwiająca załadowywanie modeli 3D w formacie OBJ stworzonym przez Wavefront Technologies.
- Dear ImGui - Napisana w C++ biblioteka umożliwiająca tworzenie lekkich interfejsów graficznych z minimalnymi zależnościami.

4.2 System aukcyjny - Frontend

4.2.1 React

Biblioteka JavaScript do budowy interfejsu użytkownika, odpowiedzialna za interaktywne komponenty strony (nawigacja, karty aukcji, formularze). Umożliwia dynamiczne renderowanie widoków bez przeładowywania całej strony.

4.2.2 React Router DOM

Biblioteka zarządzająca nawigacją i routingiem w aplikacji React. Pozwala na płynne przechodzenie między stronami (główna, aukcje, logowanie) bez odświeżania przeglądarki.

4.2.3 Tailwind CSS

Framework CSS wykorzystujący klasy użytkowe do szybkiego stylowania komponentów. Odpowiada za cały wygląd wizualny aplikacji – kolory, layout, animacje, efekty glassmorphism.

4.2.4 TW Elements

Komponenty UI oparte na Tailwind CSS. Dostarcza gotowe, interaktywne elementy interfejsu (karty, modale, powiadomienia).

4.3 System aukcyjny - Backend

4.3.1 Node.js

Środowisko uruchomieniowe JavaScript po stronie serwera. Wykonuje logikę biznesową aplikacji, obsługuje requesty HTTP i zarządza danymi użytkowników.

4.3.2 Express.js

Minimalny framework webowy dla Node.js. Odpowiada za routing API, obsługę żądań HTTP (GET, POST, PUT, DELETE) i middleware (CORS, autentykacja).

4.3.3 MongoDB

Nierelacyjna baza danych NoSQL przechowująca dokumenty w formacie JSON. Zarządza danymi użytkowników, aukcji i sesji w elastycznej, skalowalnej strukturze.

4.3.4 Mongoose

Biblioteka ODM (Object Data Modeling) dla MongoDB. Definiuje schematy danych, walidację i zapewnia wygodny interfejs do operacji bazodanowych.

4.3.5 JWT (jsonwebtoken)

Standard tworzenia tokenów uwierzytelniających. Odpowiada za bezpieczne logowanie użytkowników i utrzymywanie sesji bez przechowywania stanu na serwerze.

4.3.6 bcryptjs

Biblioteka do hashowania haseł. Zapewnia bezpieczne przechowywanie haseł użytkowników poprzez jednokierunkowe szyfrowanie z solą.

4.3.7 CORS

Middleware umożliwiający komunikację między domenami. Pozwala frontendowi (`localhost:5173`) komunikować się z backendem (inny port/domena) zgodnie z polityką bezpieczeństwa przeglądarek.

4.3.8 dotenv

Narzędzie do zarządzania zmiennymi środowiskowymi. Przechowuje wrażliwe dane (klucze API, połączenie z bazą) poza kodem źródłowym w pliku `.env`.

4.3.9 validator

Biblioteka do walidacji danych wejściowych. Sprawdza poprawność formatów (email, URL, numery telefonu) i zabezpiecza przed nieprawidłowymi danymi.

5 Wprowadzenie do renderowania 3D

Rozpoczynając pracę z zaawansowanymi technikami renderowania 3D w C++, do wyboru są trzy główne biblioteki, które pozwalają na niskopoziomową komunikację z procesorem graficznym: Vulkan, Direct3D 12 oraz Metal. Tylko Vulkan pozwala pisać kod działający na różnych platformach. DirectX jest dostępny wyłącznie na systemie Windows, natomiast Metal jest specyficzny dla ekosystemu Apple. Jednak kod napisany z użyciem tych bibliotek nie może być bezpośrednio uruchomiony w przeglądarce internetowej.

W 2016 roku firma Google zaproponowała stworzenie nowego standardu pozwalającego na komunikację z procesorem graficznym z poziomu przeglądarki internetowej. Miał być to następca WebGL, pozwalający na bardziej efektywne wykorzystanie nowoczesnych kart graficznych. W 2021 roku została opublikowana pierwsza oficjalna specyfikacja pod nazwą WebGPU. Aktualnie na koniec roku 2025 w dalszym ciągu jest to propozycja, czekająca na standaryzację, a nie oficjalny standard W3C. WebGPU jest już wspierane przez większość nowoczesnych przeglądarek internetowych, w tym Google Chrome, Microsoft Edge, Mozilla Firefox oraz Safari. Implementacja WebGPU spaja ze sobą ww. trzy biblioteki, tworząc ujednolicony interfejs programistyczny umożliwiający pisanie przenośnego kodu.

Istnieją dwie możliwości korzystania z WebGPU w aplikacjach webowych. Pierwsza z nich to użycie języka programowania JavaScript, które są natywnie obsługiwane przez przeglądarki internetowe. Druga możliwość to skorzystanie z interfejsu programistycznego w języku C udostępnianego przez dwie istniejące implementacje:

- **Dawn** – implementacja stworzona przez firmę Google, przy pomocy języka C++ - używana w przeglądarkach opartych na silniku Chromium.
- **wgpu** – implementacja stworzona przez firmę Mozilla, przy pomocy języka Rust - używana w przeglądarce Mozilla Firefox.

Następnie napisany kod należy skompilować do formatu WebAssembly. Zaletą tego podejścia jest niemalże porównywalna wydajność aplikacji z wersją natywną, gdyż kod WebAssembly jest uruchamiany bezpośrednio przez silnik przeglądarki, a nie interpretowany jak w przypadku języka JavaScript. Ponadto istniejący już kod można skompilować także do typowego dla systemu operacyjnego formatu wykonywalnego, co pozwala na uruchomienie aplikacji zarówno w przeglądarce internetowej, jak i natywnie na komputerze.

5.1 WebGPU

Najważniejsze w procesie renderowania jest stworzenie potoku renderowania, dostarczenie do niego danych, oraz przetworzenie ich używając procesora graficznego. Komunikacja z procesorem graficznym za pomocą WebGPU odbywa się na dwa sposoby. Pierwszy z nich odbywa się za pomocą obiektów reprezentujących wewnętrzne zasoby GPU. Programista ma

niewielki wpływ na ich działanie, jedynie jest w stanie wybrać konkretne, wcześniej zdefiniowane ustawienia, które są zawarte w specjalnych strukturach zwanych deskryptorami. uzupełniony deskryptor następnie przekazywany jest jako argument do odpowiedniej funkcji tworzącej zasób. Po zakończeniu działania, zasoby te należy manualnie zwolnić, aby zapobiec wyciekowi pamięci.

```
1 {
2     // ...
3
4     wgpu::SamplerDescriptor sampler_descriptor {};
5     sampler_descriptor.addressModeU = wgpu::AddressMode::Repeat;
6     sampler_descriptor.addressModeV = wgpu::AddressMode::Repeat;
7     sampler_descriptor.addressModeW = wgpu::AddressMode::Repeat;
8     sampler_descriptor.magFilter = wgpu::FilterMode::Linear;
9     sampler_descriptor.minFilter = wgpu::FilterMode::Linear;
10    sampler_descriptor.mipmapFilter =
11        wgpu::MipmapFilterMode::Nearest;
12    sampler_descriptor.lodMinClamp = 0.0F;
13    sampler_descriptor.lodMaxClamp = 8.0F;
14    sampler_descriptor.compare = wgpu::CompareFunction::Undefined;
15    sampler_descriptor.maxAnisotropy = 1U;
16
17    m_sampler = m_device.createSampler(sampler_descriptor);
18    if (m_sampler == nullptr)
19    {
20        return std::unexpected { "Failed to create sampler" };
21    }
22
23    // ...
24 }
```

Listing 5.1: Deskryptor obiektu `WGPUSampler` oraz przekazanie go do funkcji

Drugim sposobem są shadery, czyli programy wykonywane bezpośrednio na procesorze graficznym. Shadery są napisane w specjalnych językach, dla WebGPU jest to WGSL (WebGPU Shading Language). Niezbędne dla prawidłowego działania potoku renderowania są dwa rodzaje: shader wierzchołków (vertex shader) oraz shader fragmentów (fragment shader). Pierwszy z nich przetwarza dane wierzchołków, takie jak pozycja, kolor czy współrzędne tekstury. Shader fragmentów natomiast obliczają kolor i inne atrybuty każdego fragmentu, czyli danych niezbędnych do wygenerowania wartości jednego piksela. Poniżej znajduje się shader fragmentów napisany w języku WGSL. Shader próbkowuje tekstury (koloru i normalnych), rekonstruuje normalną w przestrzeni świata z użyciem macierzy TBN i miesza ją z normalną geometryczną, oblicza oświetlenie rozproszone i zwierciadlane od czterech

światła kierunkowych na podstawie współczynników k_d , k_s i twardości, wykonuje korekcję gamma i zwraca końcowy kolor z kanałem alfa z uniformu.

```
1 @fragment
2 fn fs_main(in: vertex_output) -> @location(0) vec4f {
3     let normal_map_strength = 1.0;
4     let encoded_normal = textureSample(
5         normal_texture, texture_sampler, in.uv).rgb;
6     let local_normal = encoded_normal * 2.0 - 1.0;
7     let local_to_world = mat3x3f(
8         normalize(in.tangent),
9         normalize(in.bitangent),
10        normalize(in.normal),
11    );
12    let world_normal = local_to_world * local_normal;
13    let N = mix(in.normal, world_normal, normal_map_strength);
14    var V = normalize(in.view_direction);
15
16    let base_color = textureSample(
17        base_color_texture, texture_sampler, in.uv).rgb;
18    let kd = u_lighning.kd;
19    let ks = u_lighning.ks;
20    let hardness = u_lighning.hardness;
21
22    var color = vec3f(0.0);
23    for (var i: i32 = 0; i < 4; i++)
24    {
25        let light_color = u_lighning.colors[i].xyz;
26        let L = normalize(u_lighning.directions[i].xyz);
27        let R = reflect(-L, N);
28        let diffuse = max(0.0, dot(L, N)) * light_color;
29        let RoV = max(0.0, dot(R, V));
30        let specular = pow(RoV, hardness);
31        color += base_color * (kd * diffuse) + (ks * specular);
32    }
33
34    let corrected_color = pow(color, vec3f(2.2));
35    return vec4f(corrected_color, u_my_uniforms.color.a);
36 }
```

Listing 5.2: Shadera fragmentów w WGSL

5.2 WebAssembly

W grudniu 2025 roku, 99.24% przeglądarek internetowych wspiera WebAssembly, co czyni go uniwersalnym rozwiązaniem do uruchamiania kodu wymagającego wysokiej wydajności w aplikacjach internetowych. Jednak ten format ten został stworzony jako cel kompilacji, a nie osobny język programowania. W przypadku języka C++ kod źródłowy jest kompilowany do WebAssembly przy pomocy narzędzia Emscripten. Jest to zaawansowane narzędzie, dzięki któremu przeniesiono wiele dużych aplikacji do świata internetowego. Razem z kompilatorem dostarczana także jest specjalna biblioteka, która implementuje standardowe funkcje języków C i C++ w środowisku WebAssembly. Dzięki temu możliwe jest korzystanie z takich funkcji jak alokacja pamięci, operacje na plikach czy obsługa wejścia/wyjścia. Należy jednak pamiętać, że format wyjściowy został stworzony z myślą o bezpieczeństwie i wydajności, dlatego nie wszystkie funkcje języków C i C++ są dostępne lub działają identycznie jak w natywnym środowisku. Maszyna wirtualna, w której wykonywany jest kod nie ma bezpośredniego dostępu do zasobów systemowych, co ogranicza możliwości interakcji z systemem operacyjnym. Emscripten tworzy, więc wirtualny system plików, który symuluje operacje na plikach w pamięci przeglądarki, zachowując przy tym izolację i bezpieczeństwo.

5.3 Podsumowanie

WebGPU w połączeniu z WebAssembly otwiera nowe możliwości dla programistów chcących tworzyć wydajne aplikacje 3D działające bezpośrednio w przeglądarkach internetowych. Dzięki temu możliwe jest tworzenie zaawansowanych wizualizacji, gier oraz interaktywnych aplikacji bez konieczności instalowania dodatkowego oprogramowania. Użycie tych technologii pozwala na pełne wykorzystanie potencjału nowoczesnych kart graficznych, zapewniając jednocześnie szeroką dostępność i łatwość dystrybucji aplikacji.

6 Implementacja komponentu renderującego

Implementację można podzielić w logiczny sposób na trzy główne etapy: inicjalizacja okna, inicjalizacja WebGPU, oraz pętla główna aplikacji. Etapy inicjalizacji muszą być wykonywane w określonej kolejności, dlatego naturalnie układają się w potok. Jednak inaczej niż w typowym potoku zamiast przekazywać między etapami dane, zostanie przekazywana informacja o sukcesie lub błędzie inicjalizacji poszczególnego elementu. W razie wystąpienia błędu, potok zostanie przerwany, a informacja o błędzie wyświetlona. Jest to system podobny w swoim działaniu to wyjątków, jednak dużo bardziej wydajny. Standard C++23 dostarcza do biblioteki standardowej obiekt `std::expected<T1, T2>`, który spełnia wyżej wymienioną funkcjonalność. Użyta zostanie specjalizacja `std::expected<void, std::string>`, gdzie pierwszy parametr szablonu `void` symbolizuje brak danych zwracanych w przypadku sukcesu, a drugi parametr `std::string`, przechowuje komunikat o błędzie. Jak można zauważyć na poniższym listingu, główna funkcja inicjalizująca składa się z dwóch z trzech wcześniej wymienionych etapów, ponieważ pętla główna wykonywana jest po zakończeniu poprawnej inicjalizacji.

```
1 void
2 initialize()
3 {
4     auto result = init_window()
5         .and_then([ this ]() { return init_renderer(); })
6
7     if (!result.has_value())
8     {
9         std::println(
10             std::cerr, "Error occured:\n{}", result.error());
11         std::exit(1);
12     }
13 }
```

Listing 6.1: Funkcja inicjalizująca komponent renderujący

6.1 Okno aplikacji

Pierwszym etapem jest utworzenie okna aplikacji, w którym będzie wyświetlany renderowany obraz. W tym celu wykorzystano bibliotekę GLFW, która umożliwia tworzenie okien oraz obsługę zdarzeń wejściowych w sposób niezależny od platformy. Proces inicjalizacji okna obejmuje inicjalizację biblioteki, oraz stworzenie okna. Wskaźnik na utworzone okno jest zapisywany w klasie głównej aplikacji. Opcjonalnie można też narzucić minimalny rozmiar okna lub wymusić stałą proporcję, co ułatwia utrzymanie spójnej perspektywy w czasie demonstracji.

```

1  auto
2  init_window() -> std::expected<void, std::string>
3  {
4      if (glfwInit() == GLFW_FALSE)
5      {
6          return std::unexpected { "Failed to initialize GLFW" };
7      }
8      glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
9      glfwWindowHint(GLFW_RESIZABLE, GLFW_TRUE);
10     p_window =
11         glfwCreateWindow(
12             initial_width, initial_height,
13             "INZ", nullptr, nullptr);
14     if (p_window == nullptr)
15     {
16         return std::unexpected { "Failed to create window" };
17     }
18     glfwSetWindowUserPointer(p_window, this);
19     // ...
20 }

```

Listing 6.2: Część kodu odpowiedzialna za inicjalizację okna

Reszta kodu w tej funkcji odpowiada za ustawienie asynchronicznych funkcji obsługi zdarzeń, takie jak zmiana rozmiaru okna czy obsługa myszy.

```

1  auto
2  init_window() -> std::expected<void, std::string>
3  {
4      // ...
5      glfwSetCursorPosCallback(
6          p_window,
7          [](GLFWwindow* window, double x_pos, double y_pos)
8          {
9              if (auto* app = reinterpret_cast<application*>(
10                  glfwGetWindowUserPointer(window)); app != nullptr)
11              {
12                  app->cursor_position_callback(x_pos, y_pos);
13              }
14          });
15 }

```

Listing 6.3: Dalsza część funkcji inicjalizującej okno - obsługa pozycji myszy

6.2 WebGPU

Inicjalizacja WebGPU składa się z trzynastu kroków, które muszą być wykonane w określonej kolejności, aby poprawnie skonfigurować środowisko renderujące. Poniżej przedstawiono poszczególne kroki wraz z opisem ich funkcji. W praktyce są one łączone w potok oparty na `std::expected` i operacjach `and_then`, co umożliwia natychmiastowe przerwanie przy pierwszym błędzie oraz czytelne przesłanie komunikatu o niepowodzeniu.

```
1  auto
2  init_renderer() -> std::expected<void, std::string>
3  {
4      return init_adapter()
5          .and_then([ this ] { return init_device(); })
6          .and_then([ this ] { return init_queue(); })
7          .and_then([ this ] { return init_shader_module(); })
8          .and_then([ this ] { return init_depth_texture(); })
9          .and_then([ this ] { return init_bind_group_layout(); })
10         .and_then([ this ] { return init_pipeline(); })
11         .and_then([ this ] { return init_sampler(); })
12         .and_then([ this ] { return init_textures(); })
13         .and_then([ this ] { return init_geometry(); })
14         .and_then([ this ] { return init_uniforms(); })
15         .and_then([ this ] { return init_lighting_uniforms(); })
16         .and_then([ this ] { return init_bind_group(); });
17 }
```

Listing 6.4: Funkcja inicjalizująca WebGPU, stworzona jako potok

6.2.1 Adapter

Adapter to reprezentacja fizycznego lub wirtualnego urządzenia GPU dostępnego w systemie. W tym kroku aplikacja wysyła zapytanie do systemu o dostępne adaptory, a następnie wybiera najbardziej odpowiedni na podstawie określonych kryteriów, takich jak wydajność czy obsługiwane funkcje. Aby móc uzyskać adapter, należy najpierw utworzyć instancję WebGPU (`WGPUInstance`), która stanowi globalny punkt wejścia do interfejsu WebGPU i reprezentuje implementację tego API w systemie. Następnie tworzona jest powierzchnia renderująca (`WGPUSurface`), która stanowi abstrakcję powiązaną z wcześniej zainicjalizowanym oknem GLFW i umożliwia prezentację wyrenderowanych obrazów użytkownikowi. Powierzchnia ta jest przekazywana jako parametr opcji adaptera (`compatibleSurface`), dzięki czemu wybierany adapter będzie kompatybilny z docelową powierzchnią renderowania. Dopiero po utworzeniu instancji i powierzchni możliwe jest wysłanie zapytania o adapter spełniający określone kryteria kompatybilności.

```

1  auto
2  init_adapter() -> std::expected<void, std::string>
3  {
4      m_instance = wgpuCreateInstance(nullptr);
5      if (m_instance == nullptr)
6      {
7          return std::unexpected { "Failed to initialize WebGPU" };
8      }
9      m_surface = glfwCreateWindowWGPUSurface(m_instance, p_window);
10     if (m_surface == nullptr)
11     {
12         return std::unexpected { "Failed to initialize surface" };
13     }
14     wgpu::RequestAdapterOptions adapter_options {};
15     adapter_options.compatibleSurface = m_surface;
16     m_adapter = m_instance.requestAdapter(adapter_options);
17     if (m_adapter == nullptr)
18     {
19         return std::unexpected { "Failed to create adapter" };
20     }
21     return {};
22 }

```

Listing 6.5: Funkcja inicjalizująca obiekt adapter

6.2.2 Urządzenie

Urządzenie to logiczne przedstawienie wybranego adaptera, które umożliwia tworzenie zasobów procesora graficznego oraz wykonywanie operacji renderujących. W tym kroku aplikacja tworzy obiekt `WGPUDevice`, który będzie używany do komunikacji z procesorem graficznym. Przed jego utworzeniem z adaptera odczytywane są wspierane limity (`WGPULimits`), a następnie definiowany jest zestaw wymaganych limitów przekazywany w deskrytorze urządzenia. Limity te określają oczekiwania aplikacji względem: organizacji geometrii (np. liczby i struktury atrybutów oraz buforów wierzchołków), poprawnych przesunięć dla operacji na pamięci, przepływu danych między etapami shadera (uniformy), a także modelu powiązań zasobów. Obejmują również ograniczenia dotyczące tekstur i samplerów (wymiany oraz liczebność na etap), tak aby odpowiadały realnym wymaganiom potoku i używanych materiałów. Dzięki zastosowaniu limitów urządzenie jest tworzone tylko wtedy, gdy implementacja WebGPU gwarantuje parametry wystarczające do bezpiecznego i wydajnego działania. W `WGPUDeviceDescriptor` ustawiane są ponadto etykiety, wskaźnik do wymaganych limitów, opis domyślnej kolejki oraz funkcje zwrotne dla utraty urządzenia i błędów nieprzechwyco-

nych, które służą diagnostyce. Po utworzeniu urządzenia dobierany jest format powierzchni (BGRA8Unorm) i wykonywana jest konfiguracja (`configure_surface()`), aby zapewnić poprawne wyświetlanie na utworzonej wcześniej powierzchni.

```
1  auto
2  init_device() -> std::expected<void, std::string>
3  {
4      auto required_limits = get_limits(m_adapter);
5      wgpu::DeviceDescriptor device_descriptor {};
6      // ...
7
8      m_device = m_adapter.requestDevice(device_descriptor);
9      if (m_device == nullptr)
10     {
11         return std::unexpected { "Failed to create device" };
12     }
13     m_surface_format = wgpu::TextureFormat::BGRA8Unorm;
14     configure_surface();
15     return {};
16 }
```

Listing 6.6: Funkcja inicjalizująca obiekt device

6.2.3 Zasoby tworzone przez urządzenie

Po utworzeniu urządzenia (WGPUDevice) większość pozostałych zasobów procesora graficznego jest tworzona za jego pomocą. Proces inicjalizacji tych zasobów podąża za wspólnym wzorcem: najpierw tworzony jest obiekt deskryptora, który zawiera szczegółową konfigurację zasobu, następnie wywoływana jest odpowiednia funkcja, która zwraca gotowy zasób. Takie podejście zapewnia spójność i czytelność kodu, a także ułatwia diagnostykę problemów poprzez sprawdzenie, czy zwrócony wskaźnik nie jest pustym wskaźnikiem.

Kolejka poleceń Kolejka poleceń (WGPUQueue) jest uzyskiwana bezpośrednio z urządzenia bez potrzeby tworzenia deskryptora. Dzięki kolejce kod aplikacji wykonywany na procesorze komputera może wysyłać polecenia do procesora graficznego w sposób asynchroniczny.

Moduł shaderów Moduł shaderów zawiera skompilowany kod shaderów napisanych w języku WGSL. Shadery definiują sposób przetwarzania wierzchołków (vertex shader) oraz obliczania koloru pikseli (fragment shader). W implementacji kod shaderu jest ładowany z pliku za pomocą funkcji pomocniczej `load::shader_module()`, która wewnętrznie wykorzystuje `device.createShaderModule()`.

Tekstura głębi Tekstura głębi to specjalny zasób do przechowywania informacji o odległości każdego piksela od kamery w scenie 3D. Jest używana podczas testów głębi w potoku renderowania, aby określić, które fragmenty są widoczne, a które zasłonięte przez bliższe obiekty. W deskrytorze tekstury kluczowe są parametry: format (Depth24Plus - 24-bitowa precyzja głębi), usage (RenderAttachment - użycie jako załącznik renderowania), oraz wymiary odpowiadające wymiarom okna. Po utworzeniu tekstury, tworzony jest także widok tekstury (TextureView) poprzez wywołanie `texture.createView()`, który jest używany w potoku renderowania.

```
1  auto
2  init_depth_texture() -> std::expected<void, std::string>
3  {
4      wgpu::TextureDescriptor depth_texture_descriptor = //...
5      m_depth_texture =
6          m_device.createTexture(depth_texture_descriptor);
7      if (m_depth_texture == nullptr)
8      {
9          return std::unexpected { "Failed to create depth texture" };
10     }
11     auto depth_texture_view_descriptor = //...
12     m_depth_texture_view =
13         m_depth_texture.createView(depth_texture_view_descriptor);
14     if (m_depth_texture_view == nullptr)
15     {
16         return
17             std::unexpected { "Failed to create depth texture view" };
18     }
19     return {};
20 }
```

Listing 6.7: Funkcja inicjalizująca obiekt depth texture

Układ grupy powiązań Układ grupy powiązań definiuje interfejs między zbiorem zasobów a ich dostępnością w poszczególnych etapach shadera. W deskrytorze określone są typy zasobów (bufory uniformów, tekstury, samplery), ich indeksy powiązań oraz widoczność w shaderach wierzchołków i fragmentów. Jest to schemat określający, jakie zasoby będą dostępne dla shaderów, ale nie zawiera jeszcze konkretnych instancji tych zasobów.

Układ potoku i potok renderowania Potok renderowania jest kluczowym elementem procesu renderowania, definiującym wszystkie etapy przetwarzania danych graficznych. Według standardu WebGPU obejmuje on następujące etapy: pobieranie wierzchołków (vertex fetch),

shader wierzchołków (vertex shader), składanie prymitywów (primitive assembly), rasteryzacja (rasterization), shader fragmentów (fragment shader), test szablonu i operacje (stencil test and operation), test głębi i zapis (depth test and write), oraz scalanie wyjścia (output merging).

Tworzenie potoku renderowania jest bardziej złożone niż innych zasobów i wymaga wielu struktur konfiguracyjnych. Najpierw tworzony jest układ potoku (WGPUPipelineLayout), który określa ogólną organizację zasobów dostępnych dla shaderów, bazując na wcześniej utworzonym obiekcie WGPUBindGroupLayout. Następnie wypełniany jest deskryptor potoku renderowania (WGPURenderPipelineDescriptor), który zawiera konfigurację wszystkich wyżej wymienionych etapów. W implementacji poszczególne etapy konfigurowane są przez dedykowane funkcje pomocnicze.

```
1 auto
2 init_pipeline() -> std::expected<void, std::string>
3 {
4     auto vertex_attribs = get_vertex_attributes();
5     auto vertex_buffer_layout =
6         get_vertex_buffer_layout(vertex_attribs);
7     wgpu::RenderPipelineDescriptor pipeline_desc {};
8     // Fill vertex and primitive stage
9     wgpu::FragmentState fragment_state {};
10    wgpu::BlendState blend_state {};
11    wgpu::ColorTargetState color_target {};
12    // Fill fragment stage
13    wgpu::DepthStencilState depth_stencil_state = wgpu::Default;
14    // Fill depth-stencil and multisampling stage
15    auto pipeline_layout_desc = get_pipeline_layout_desc();
16    m_layout =
17        m_device.createPipelineLayout(pipeline_layout_desc);
18    pipeline_desc.layout = m_layout;
19    m_pipeline = m_device.createRenderPipeline(pipeline_desc);
20    if (m_pipeline == nullptr)
21    {
22        return std::unexpected("Failed to create pipeline");
23    }
24    return {};
25 }
```

Listing 6.8: Funkcja inicjalizująca obiekt pipeline

Sampler Sampler koduje informacje dotyczące filtrowania i adresowania tekstur, które są wykorzystywane w shaderach podczas próbkowania (sampling) tekstur. W deskrypcorze okre-

ślane są tryby adresowania dla każdej osi (Repeat - powtarzanie tekstury), filtry powiększania i pomniejszania (Linear - interpolacja liniowa dla gładszego wyglądu), oraz parametry poziomów szczegółowości (LOD).

Bufory Bufor to blok pamięci procesora graficznego, który może przechowywać różnorodne dane wykorzystywane w operacjach graficznych. Wszystkie buforory tworzone są według tego samego wzorca: w deskrytorze określany jest rozmiar bufora, sposób jego użycia (usage) oraz czy ma być mapowany przy tworzeniu. W implementacji używane są trzy rodzaje buforów:

- **Bufor wierzchołków** - przechowuje dane geometrii modeli 3D (pozycje, normalne, współrzędne tekstur). Dane są najpierw ładowane z pliku do pamięci CPU, a następnie przesyłane do bufora procesora graficznego funkcją `queue.writeBuffer()`. Parametr usage: CopyDst | Vertex.
- **Bufor uniformów transformacji** - przechowuje macierze transformacji (model, widok, projekcja), pozycję kamery oraz dodatkowe parametry renderowania. Jest aktualizowany w każdej klatce, gdy zmienia się perspektywa kamery lub rotacja modelu. Parametr usage: CopyDst | Uniform.
- **Bufor uniformów oświetlenia** - zawiera informacje o źródłach światła w scenie: kierunki światła, ich kolory oraz parametry modelu oświetlenia (kd, ks, hardness). Parametr usage: CopyDst | Uniform.

Tekstury materiałów Tekstury to wielowymiarowe tablice danych reprezentujące informacje wizualne, takie jak kolory czy mapy normalnych. W implementacji tekstury są ładowane z plików graficznych za pomocą funkcji pomocniczej `load::texture()`, która wewnętrznie używa `device.createTexture()` oraz tworzy odpowiedni widok tekstury.

Grupa powiązań Grupa powiązań łączy konkretne instancje zasobów (bufory, tekstury, samplery) z ich deklaracjami w układzie grupy powiązań. W deskrytorze określany jest wcześniej utworzony obiekt `WGPUBindGroupLayout` oraz tablica wpisów (`WGPUBindGroupEntry`), z których każdy przypisuje konkretny zasób do odpowiedniego indeksu powiązania, co umożliwia shaderom dostęp do tych zasobów podczas renderowania.

6.3 Pętla główna

W pętli głównej wykonywane są ciągłe aktualizacje sceny oraz renderowanie obrazu. Pętla ta działa do momentu zamknięcia okna aplikacji. W każdej iteracji pętli wykonywane są następujące kroki:

1. Obsługa zdarzeń okna

2. Aktualizacja danych uniformów
3. Zmiana rozmiaru powierzchni renderującej w przypadku zmiany rozmiaru okna
4. Pobranie następnego widoku tekstury powierzchni do renderowania i natychmiastowe przerwanie iteracji jeśli takowego nie ma.
5. Utworzenie enkodera poleceń używanego do nagrywania poleceń procesora graficznego.
6. Stworzenie obiektu render pass. Każda instancja tego obiektu definiuje zestaw zasobów obrazów, zwanych załącznikami, wykorzystywanych podczas renderowania.
7. Ustawienie wcześniej zainicjalizowanych elementów: potoku renderowania, bufora wierzchołków oraz powiązań uniformów.
8. Wykonanie polecenia rysowania.
9. Zakończenie działania obiektu render pass.
10. Sfinalizowanie nagranych poleceń: przesłanie poleceń do kolejki procesora graficznego.

7 Implementacja wyglądu serwisu aukcyjnego

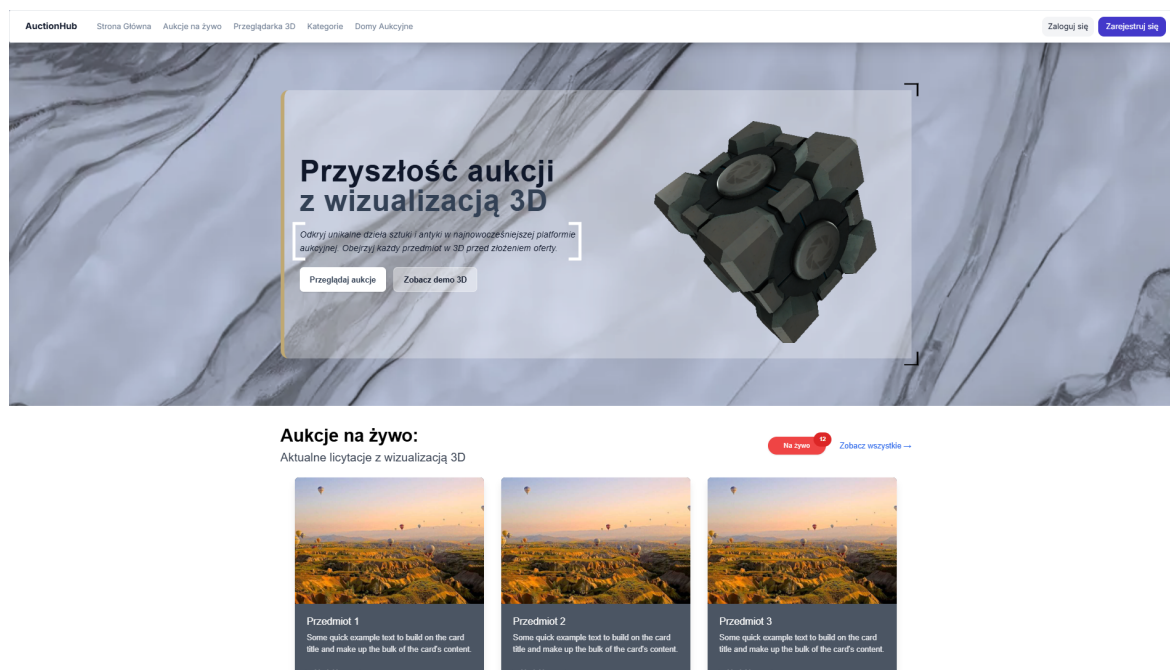
7.1 Koncepcja interfejsu użytkownika

Frontend aplikacji został zaprojektowany z myślą o nowoczesnym, minimalistycznym designie, który kładzie nacisk na prezentację wizualizacji 3D oferowanych obiektów. Interfejs wykorzystuje technologie React oraz Tailwind CSS, co pozwoliło na stworzenie responsywnego i intuicyjnego środowiska aukcyjnego z płynną nawigacją oraz estetycznym układem elementów.

7.2 Prezentacja poszczególnych widoków

7.2.1 Strona główna

Strona główna została zaprojektowana jako wizytówka całej platformy, z charakterystycznym hero section prezentującym główną wartość aplikacji - możliwość przeglądania aukcji z wizualizacją 3D. W centralnej części ekranu znajduje się transparentny panel z nagłówkiem „Przyszłość aukcji z wizualizacją 3D” oraz krótkim opisem: „Odkryj unikalne przedmioty artystyczne w innowacyjny sposób. Zobacz aukcje z wizualizacją 3D zamiast zwykłych zdjęć.”



Rysunek 7.1: Widok strony głównej aplikacji AuctionHub

Kluczowym elementem designu jest umieszczenie interaktywnego modelu 3D bezpośrednio na stronie głównej, co natychmiast komunikuje użytkownikowi unikalną funkcjonalność platformy. Panel zawiera dwa przyciski: „Przeglądaj aukcje” oraz „Zobacz demo 3D”, co pozwala użytkownikom od razu zaangażować się w eksplorację platformy.

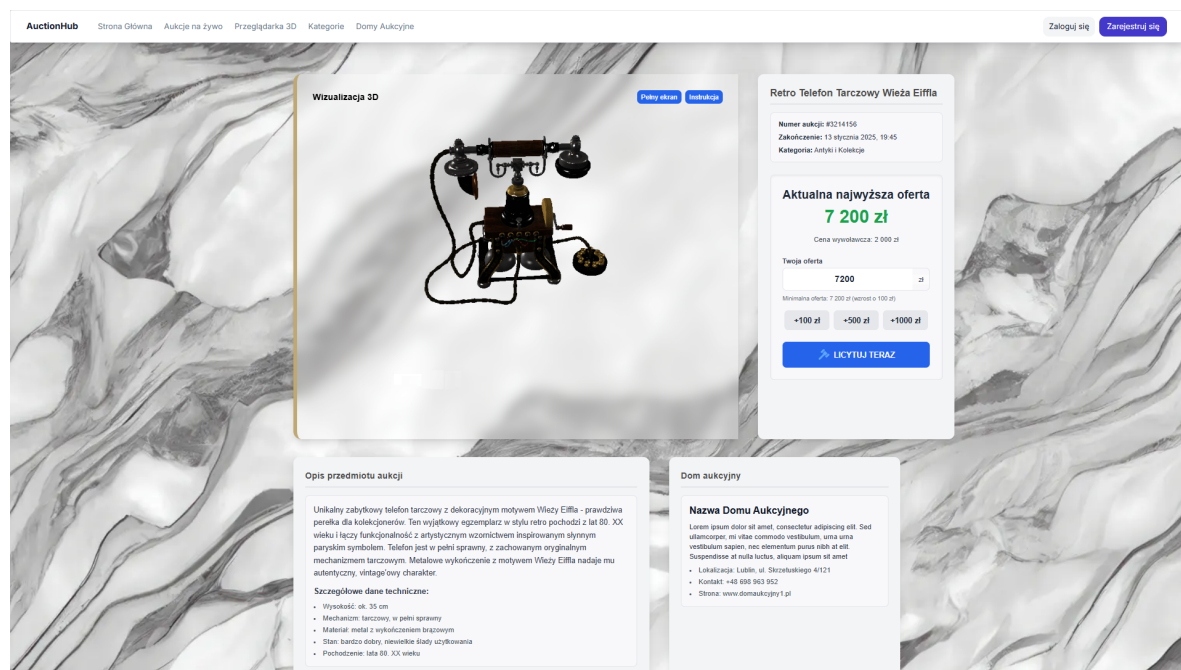
Poniżej hero section znajduje się sekcja „Aukcje na żywo” z aktualnie trwającymi licytacjami. Wykorzystano tutaj system kart z wizualizacjami 3D aukcji. Każda karta zawiera

miniaturę wizualizacji, tytuł aukcji oraz krótki opis. Sekcja posiada również czerwony badge „Na żywo” z licznikiem aktywnych aukcji oraz link „Zobacz wszystkie” dla pełnego dostępu do ofert.

Tło strony wykorzystuje delikatną teksturę marmuru w odcieniach szarości, co nadaje platformie elegancki, luksusowy charakter odpowiedni dla aukcji przedmiotów artystycznych.

7.2.2 Strona szczegółów aukcji

Widok szczegółowy aukcji dzieli ekran na dwie główne sekcje. Po lewej stronie znajduje się duży panel z wizualizacją 3D obiektu. Prawa kolumna zawiera pełne informacje o aukcji. Pod panelem wizualizacji znajdują się dwie sekcje informacyjne.



Rysunek 7.2: Widok szczegółów aukcji z wizualizacją 3D

7.2.3 System nawigacji

Oba widoki zawierają identyczny, minimalistyczny pasek nawigacji z logo „AuctionHub” po lewej stronie oraz menu zawierające linki: „Strona Główna”, „Aukcje na żywo”, „Przeglądanie 3D”, „Kategorie”, „Domy Aukcyjne”. Po prawej stronie paska znajdują się przyciski „Zaloguj się” oraz wyróżniony przycisk „Zarejestruj się”.

7.2.4 Strony logowania i rejestracji

Aplikacja zawiera również dedykowane widoki do autoryzacji użytkowników. Strony logowania i rejestracji utrzymane są w spójnej stylistyce z resztą aplikacji, wykorzystując to

samo tło marmurowe oraz transparentne panele formularzy. Formularze zawierają standardowe pola (email, hasło) oraz przyciski akcji w charakterystycznym niebieskim kolorze. Design tych stron jest minimalistyczny i skupiony na funkcjonalności, zapewniając użytkownikom szybki i bezproblemowy proces uwierzytelnienia.

AuctionHub Strona Główna Aukcje na żywo Przeglądarka 3D Kategorie Domy Aukcyjne Zaloguj się Zarejestruj się

Witaj ponownie
Zaloguj się do swojego konta AuctionHub

Email lub Nazwa użytkownika

Hasło

☐ Zapamiętaj mnie [Zapomniałeś hasła?](#)

Nie masz jeszcze konta? [Zarejestruj się](#)

AuctionHub
Pierwsza platforma aukcyjna z
wizualizacją 3D w Polsce. Łączymy
tradycję z nowoczesną technologią.

Aukcje
Aktualne aukcje
Archiwum
Kategorie

Dla domów aukcyjnych
Dołącz do nas
Panel zarządzania
Wsparcie techniczne

Pomoc i kontakt
FAQ
Kontakt
Polityka prywatności

© 2025 AuctionHub. Wszystkie prawa zastrzeżone. Regulamin Prywatność Cookies

Rysunek 7.3: Widok strony logowania

AuctionHub Strona Główna Aukcje na żywo Przeglądarka 3D Kategorie Domy Aukcyjne Zaloguj się Zarejestruj się

Dołącz do AuctionHub
Utwórz konto i zacznij licytować

Nazwa użytkownika *

Min. 3 znaki, tylko litery, cyfry, _ i -

Adres e-mail *

Imię Nazwisko

Numer telefonu

Hasło * Potwierdź hasło *
Min. 8 znaków

Masz już konto? [Zaloguj się](#)

AuctionHub
Pierwsza platforma aukcyjna z
wizualizacją 3D w Polsce. Łączymy
tradycję z nowoczesną technologią.

Aukcje
Aktualne aukcje
Archiwum
Kategorie

Dla domów aukcyjnych
Dołącz do nas
Panel zarządzania
Wsparcie techniczne

Pomoc i kontakt
FAQ
Kontakt
Polityka prywatności

© 2025 AuctionHub. Wszystkie prawa zastrzeżone. Regulamin Prywatność Cookies

Rysunek 7.4: Widok strony rejestracji

8 Implementacja Bazy Danych oraz Logiki Biznesowej

8.1 Model Aukcji (Auction)

Model Auction stanowi rdzeń systemu aukcyjnego i zawiera logikę biznesową związaną z zarządzaniem aukcjami.

Pole	Typ	Opis
title	String	Tytuł aukcji (3-100 znaków, wymagane)
description	String	Opis aukcji (10-5000 znaków, wymagane)
category	Enum	12 kategorii: Sztuka, Antyki, Biżuteria, Monety i Banknoty, Książki, Militaria, Meble, Porcelana i Ceramika, Zegarki, Instrumenty Muzyczne, Elektronika, Inne
images	Array[String]	Tablica URLi obrazów
model3D	String	Opcjonalny model 3D (formaty: .obj, .gltf, .glb, .fbx)
startingPrice	Number	Cena wywoławcza (wymagana, ≥ 0)
currentPrice	Number	Aktualna cena aukcji
bidIncrement	Number	Minimalne podwyższenie (domyślnie: 100)
buyNowPrice	Number	Opcjonalna cena „Kup Teraz”
seller	ObjectId	Referencja do modelu User (wymagane, indeksowane)
startTime	Date	Data rozpoczęcia aukcji
endTime	Date	Data zakończenia aukcji
status	Enum	Status: draft, active, completed, cancelled
winnerId	ObjectId	Zwycięzca aukcji (referencja do User)
condition	Enum	Stan przedmiotu: Nowy, Bardzo dobry, Dobry, Zadowolający, Do renowacji
timestamps	Date	Automatyczne pola: createdAt, updatedAt

Model zawiera również pola pomocnicze: totalBids, watchers, watchersCount, views, featured, isDeleted.

Indeksy Bazy Danych System wykorzystuje **9 strategicznych indeksów** dla optymalizacji zapytań, w tym indeksy złożone (seller + status, endTime + status, category + status) oraz indeks tekstowy na polach title + description dla full-text search.

Kluczowe metody

- **placeBid(bidderId, amount)** — główna metoda biznesowa do składania licytacji. Waliduje status aukcji, czas, własność i minimalną kwotę. Aktualizuje currentPrice,

winnerId oraz totalBids.

- **complete()** — zamyka aukcję, ustawia zwycięzcę i aktualizuje statystyki użytkowników.
- **closeExpiredAuctions()** — metoda statyczna batch processing do automatycznego zamykania wygasłych aukcji, kompatybilna z cron jobs.

Pozostałe metody (cancel, softDelete, addWatcher/removeWatcher, incrementViews) działają analogicznie — modyfikują odpowiednie pola modelu z walidacją.

8.2 Model Licytacji (Bid)

Model Bid reprezentuje pojedynczą licytację w systemie.

Pole	Typ	Opis
auction	ObjectId	Referencja do Auction (wymagane, indeksowane)
bidder	ObjectId	Referencja do User (wymagane, indeksowane)
amount	Number	Kwota licytacji (wymagane, ≥ 0)
isWinning	Boolean	Czy to aktualnie wygrywająca licytacja
status	Enum	Status: active, outbid, won, lost, cancelled

Kluczowe metody

- **markLostBids(auctionId, winnerId)** — batch update oznaczający wszystkie licytacje (oprócz zwycięskiej) jako przegrane po zakończeniu aukcji.
- **getAuctionBids/getUserBids** — metody pobierające historię licytacji z paginacją i filtrowaniem. Działają analogicznie dla aukcji i użytkownika.

8.3 Model Użytkownika (User)

Model User zawiera dane użytkownika oraz system autentykacji.

Pole	Typ	Opis
username	String	Nazwa użytkownika (unikalne)
email	String	Adres email (unikalne)
password	String	Hasło (bcrypt hash)
role	Enum	Rola: user, admin
isActive	Boolean	Czy konto jest aktywne
stats	Object	Statystyki: totalAuctionsCreated, totalAuctionsWon, totalBidsPlaced, totalItemsSold

Model zawiera również pola profilu (`firstName`, `lastName`, `phone`, `avatar`, `address`) oraz tokeny autentykacji.

Zabezpieczenia

- Hasło hashowane przez **bcrypt** (cost factor: 12)
- Pole `password` wyłączone z domyślnych queries
- JWT token z 7-dniowym czasem wygaśnięcia

8.4 Strategia Walidacji

System implementuje **wielopoziomową walidację**:

1. **Schema (Mongoose)** — typy danych, wymagalność pól, ograniczenia długości, wartości enum
2. **Model (Business Logic)** — metody instancji, middleware hooks, warunki złożone
3. **Kontroler (Authorization)** — weryfikacja własności zasobów, sprawdzanie ról

8.5 Relacje między modelami

- **User** → **Auction** (1:N) — pole `seller`
- **User** → **Bid** (1:N) — pole `bidder`
- **Auction** → **Bid** (1:N) — pole `auction`
- **Auction** ↔ **User** (N:M) — tablica `watchers`

8.6 Implementacja REST API

API implementuje standardy **RESTful**: GET (pobieranie), POST (tworzenie), PATCH (aktualizacja), DELETE (usuwanie).

8.6.1 Middleware Autentykacji

- **protect** — ekstrakcja i weryfikacja JWT z nagłówka `Authorization: Bearer <token>`, pobranie użytkownika z bazy, sprawdzenie aktywności konta
- **restrictTo(...roles)** — kontrola dostępu RBAC, zwraca 403 Forbidden dla nieuprawnionych ról

8.6.2 Kluczowe endpointy API

Pobieranie aukcji (GET /api/auctions) Endpoint publiczny z filtrowaniem (category, minPrice, maxPrice, status), sortowaniem i paginacją. Analogicznie działa GET /api/auctions/search z wyszukiwaniem full-text oraz GET /api/auctions/:id dla pojedynczej aukcji.

Tworzenie aukcji (POST /api/auctions) Endpoint chroniony (protect). Wymagane pola: title, description, category, startingPrice, bidIncrement, startTime, endTime, condition. Opcjonalne: images, model3D, reservePrice, buyNowPrice. Automatycznie przypisuje seller i aktualizuje statystyki.

Składanie licytacji (POST /api/auctions/:id/bid) Endpoint chroniony. Waliduje: aktywność aukcji, minimalną kwotę ($\geq \text{currentPrice} + \text{bidIncrement}$), brak własności aukcji. Post-save hook automatycznie aktualizuje cenę aukcji, winnerId oraz oznacza poprzednie bidy jako outbid.

Pozostałe endpointy Endpointy PATCH /api/auctions/:id i DELETE /api/auctions/:id działają analogicznie z walidacją własności i ograniczeniami edycji w zależności od statusu aukcji. Endpointy watchlist (POST/DELETE /api/auctions/:id/watch) oraz historia licytacji (GET /api/bids/me, GET /api/auctions/:id/bids) działają w podobny sposób z odpowiednią autoryzacją.

8.6.3 Format odpowiedzi

Sukces:

```
1 {  
2   "success": true,  
3   "data": { ... },  
4   "results": 10,
```

```
5   "totalPages": 5,  
6   "currentPage": 1  
7 }
```

Błąd:

```
1 {  
2   "success": false,  
3   "error": {  
4     "message": "Opis błędu",  
5     "statusCode": 400  
6   }  
7 }
```

Kody statusów: 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 500 (Internal Server Error).

8.6.4 Przepływ danych — Złożenie licytacji

1. Użytkownik: `POST /api/auctions/:id/bid { amount }`
2. Kontroler waliduje żądanie HTTP
3. `Auction.placeBid()` — walidacja biznesowa (aktywność, kwota, własność)
4. Utworzenie dokumentu Bid
5. Post-save hook aktualizuje: `Auction.currentPrice`, `winnerId`, poprzednie bity na `outbid`
6. Zwrot zaaktualizowanej licytacji

Pozostałe scenariusze (zakończenie aukcji, wyszukiwanie) działają analogicznie z wykorzystaniem odpowiednich metod modeli.

9 Wymagania sprzętowe

Finalna aplikacja działa bez problemów na wszystkich przeglądarkach wspierających technologię WebGPU. Poniżej przedstawiono listę kompatybilnych przeglądarek wraz z minimalnymi wymaganiami:

9.1 Przeglądarki desktopowe

- **Google Chrome** — wersja 113 i kolejne

- **Microsoft Edge** — wersja 113 i kolejne
- **Opera** — wersja 99 i kolejne
- **Safari** — wersja 26.0 i kolejne, wyłącznie na systemie macOS 26 Tahoe lub późniejszych
- **Mozilla Firefox** — wersja 141 i kolejne z następującymi zastrzeżeniami:
 - Windows: dostępne bez dodatkowej konfiguracji od wersji 141
 - macOS 26 Tahoe lub późniejsze: dostępne bez konfiguracji od wersji 145
 - Pozostałe systemy: wymagane włączenie flagi `dom.webgpu.enabled` w ustawieniach przeglądarki

9.2 Przeglądarki mobilne

- **Chrome dla Androida** — wersja 142 i kolejne
- **Safari oraz inne przeglądarki na iOS** — wersja 26.0 i kolejne
- **Samsung Internet** — wersja 29 i kolejne
- **Opera dla Androida** — wersja 80 i kolejne

10 Integracja

Proces integracji obejmuje kilka warstw technicznych: kompilację kodu natywnego do formatu WebAssembly przy użyciu kompilatora Emscripten, zaprojektowanie architektury komponentów React umożliwiającej efektywne zarządzanie pojedynczą instancją komponentu renderującego oraz implementację mechanizmów komunikacji między kodem JavaScript a skompilowanym kodem C++.

10.1 Kompilacja do WebAssembly

Kompilacja komponentu renderującego do formatu WebAssembly wymaga odpowiedniej konfiguracji systemu budowania CMake oraz użycia toolchaina Emscripten. Proces ten przekształca kod źródłowy C++ w pliki wykonywalne w środowisku przeglądarki internetowej.

10.1.1 Konfiguracja CMake dla Emscripten

Konfiguracja projektu dla Emscripten wymaga zdefiniowania specyficznych flag kompilacji, które umożliwiają integrację z WebGPU oraz eksport funkcji dostępnych z poziomu JavaScript. Najważniejsze flagi kompilacji to:

- `-sUSE_WEBGPU=1` - włączenie wsparcia dla API WebGPU w skompilowanym module,
- `-sASYNCIFY` - umożliwienie asynchronicznego wykonywania kodu, niezbędne dla operacji WebGPU,
- `-sEXPORTED_FUNCTIONS` - lista funkcji C++ eksportowanych do JavaScript,
- `-sEXPORTED_RUNTIME_METHODS` - metody runtime Emscripten dostępne z poziomu JavaScript,
- `-sFETCH` - wsparcie dla asynchronicznego pobierania zasobów,
- `-sALLOW_MEMORY_GROWTH=1` - dynamiczne rozszerzanie pamięci WebAssembly.

10.1.2 Generowane pliki wyjściowe

Proces kompilacji generuje dwa główne pliki:

- `main.wasm` - binarny moduł WebAssembly zawierający skompilowany kod C++,
- `main.js` - kod JavaScript pozwalający na wywoływanie formatu WebAssembly. Odpowiedzialny za inicjalizację modułu, zarządzanie pamięcią oraz mostkowanie wywołań między JavaScript a WebAssembly.

Plik `main.js` definiuje globalny obiekt `Module`, który stanowi interfejs komunikacji z kodem `WebAssembly`. Obiekt ten zawiera atrybuty, które kod wygenerowany przez `Emscripten` wywołuje w różnych momentach swojego działania.

10.2 Architektura integracji w React

Integracja komponentu `WebGPU` z aplikacją `React` opiera się na wzorcu `Provider`, który zapewnia współdzielenie pojedynczej instancji komponentu renderującego między wieloma komponentami interfejsu użytkownika. Architektura ta rozwiązuje problem ponownej inicjalizacji kosztownego kontekstu `WebGPU` przy nawigacji między widokami aplikacji.

10.2.1 Wzorzec Provider

Komponent `WebGPUCanvasProvider` implementuje wzorzec `Provider` z `React Context API`. Działa jako singleton zarządzający cyklem życia modułu `WebAssembly` oraz elementem `canvas`, na którym wykonywane jest renderowanie. `Provider` opakowuje całą aplikację, zapewniając dostęp do API komponentu renderującego z dowolnego miejsca w drzewie komponentów.

```
1 export function WebGPUCanvasProvider({ children }) {
2   const [moduleReady, setModuleReady] = useState(false)
3   const [isLoading, setIsLoading] = useState(true)
4   const [error, setError] = useState(null)
5   const [currentModel, setCurrentModel] = useState('fourareen')
6
7   // ... logika zarządzania modulem WASM
8   return (
9     <WebGPUCanvasContext.Provider value={contextValue}>
10      {children}
11      {/* Wspoldzielony canvas renderowany przez Portal */}
12    </WebGPUCanvasContext.Provider>
13  )
14 }
```

Listing 10.1: Struktura komponentu `Provider`

10.2.2 Hooki dostępu do API

Dostęp do funkcjonalności komponentu renderującego z komponentów potomnych realizowany jest przez dedykowane hooki `React`:

- `useWebGPUCanvas()` - publiczny hook zwracający API do kontroli komponentu renderującego (zmiana modelu, status ładowania, informacje o błędach),

- `useWebGPUCanvasInternal()` - wewnętrzny hook używany przez komponent `WebGPUCanvas` do zarządzania slotami wyświetlania.

```

1  const publicApi = {
2    moduleReady,    // czy moduł WASM jest zainicjalizowany
3    isLoading,      // czy trwa ładowanie
4    error,          // komunikat błędu (jesli wystąpił)
5    currentModel,   // nazwa aktualnie wyświetlanego modelu
6    changeModel,    // funkcja zmiany modelu 3D
7    modelNames      // lista dostępnych modeli
8  }

```

Listing 10.2: Publiczne API komponentu renderującego

10.3 Mechanizm React Portals

Kluczowym elementem architektury jest wykorzystanie mechanizmu React Portals do dynamicznego przenoszenia elementu canvas między różnymi miejscami w strukturze DOM. Pozwala to na wyświetlanie tego samego komponentu renderującego 3D w różnych sekcjach aplikacji bez konieczności reinicjalizacji kontekstu WebGPU.

10.3.1 Zasada działania

React Portal umożliwia renderowanie komponentu potomnego do węzła DOM znajdującego się poza hierarchią rodzica. W przypadku integracji WebGPU, współdzielony canvas jest renderowany przez Portal do aktualnie aktywnego „slotu” - kontenera DOM wskazanego przez komponent `WebGPUCanvas`.

```

1  {hostNode
2    ? createPortal(
3      <SharedWebGPUCanvas
4        options={presentationOptions}
5        isLoading={isLoading}
6        error={error}
7        moduleReady={moduleReady}
8      />,
9      hostNode
10    )
11  : null}

```

Listing 10.3: Renderowanie canvas przez Portal

10.3.2 Zarządzanie slotami

System slotów pozwala komponentom WebGPUCanvas rezerwować miejsce wyświetlania komponentu renderującego. Gdy komponent jest montowany, rejestruje swój kontener jako aktywny slot. Gdy jest odmontowywany, slot jest zwalniany, a canvas może zostać przeniesiony do innego miejsca lub ukryty w fallbackowym kontenerze.

Funkcje zarządzania slotami:

- `attachSlot(slotId, container, options)` - rejestracja nowego slotu z opcjami wyświetlania (wymiary, model),
- `detachSlot(slotId)` - zwolnienie slotu przy odmontowaniu komponentu.

10.4 Ładowanie modułu WebAssembly

Inicjalizacja modułu WebAssembly odbywa się asynchronicznie podczas pierwszego montowania komponentu canvas. Proces ten obejmuje wstrzyknięcie skryptu `main.js`, konfigurację obiektu `Module` oraz obsługę stanów ładowania.

10.4.1 Konfiguracja obiektu Module

Przed załadowaniem skryptu Emscripten, konfigurowany jest globalny obiekt `window.Module` z callbackami obsługującymi zdarzenia cyklu życia:

```
1  const moduleConfig = {
2    canvas: canvasRef.current,
3
4    onRuntimeInitialized: () => {
5      setModuleReady(true)
6      setIsLoading(false)
7    },
8
9    onAbort: (what) => {
10     setError('WebAssembly initialization failed')
11     setIsLoading(false)
12   },
13
14   print: (text) => console.log('WASM:', text),
15   printErr: (text) => console.error('WASM Error:', text)
16 }
17
18 window.Module = moduleConfig
```

Listing 10.4: Konfiguracja obiektu Module

10.4.2 Asynchroniczne wstrzykiwanie skryptu

Skrypt `main.js` jest dynamicznie dodawany do dokumentu HTML. Mechanizm zapewnia, że skrypt jest ładowany tylko raz, niezależnie od liczby montowań komponentu:

```
1  if (!globalState.scriptAppended) {
2    const script = document.createElement('script')
3    script.src = '/main.js'
4    script.async = true
5    script.onerror = () => {
6      setError('Failed to load WebAssembly runtime')
7      setIsLoading(false)
8    }
9
10   document.body.appendChild(script)
11   globalState.scriptAppended = true
12 }
```

Listing 10.5: Dynamiczne ładowanie skryptu Emscripten

10.5 Wirtualny system plików

Emscripten udostępnia wirtualny system plików (VFS), który emuluje operacje plikowe w środowisku przeglądarki. Integracja wykorzystuje IDBFS - backend oparty na IndexedDB - do persystentnego przechowywania zasobów między sesjami użytkownika.

10.5.1 Montowanie systemu plików

Podczas fazy `preRun` konfigurowane są punkty montowania dla persystentnych danych:

```
1  moduleConfig.preRun.push(() => {
2    const FS = window.FS
3    const IDBFS = window.IDBFS
4
5    // Katalog na baze danych modeli
6    FS.mkdir('/database')
7    FS.mount(IDBFS, {}, '/database')
8
9    // Katalog na cache zasobow
10   FS.mkdir('/resources_cache')
11   FS.mount(IDBFS, {}, '/resources_cache')
12 })
```

Listing 10.6: Konfiguracja IDBFS

Synchronizacja z IndexedDB wykonywana jest przez funkcję `FS.syncfs()`, która może działać w trybie odczytu (z IndexedDB do VFS) lub zapisu (z VFS do IndexedDB).

10.6 Komunikacja JavaScript-C++

Komunikacja między warstwą JavaScript a skompilowanym kodem C++ odbywa się przez funkcje eksportowane z modułu WebAssembly. Funkcje te są dostępne jako metody obiektu `window.Module`.

10.6.1 Eksportowane funkcje

Komponent renderujący eksportuje następujące funkcje:

- `change_model(modelName)` - zmiana wyświetlanego modelu 3D,
- `resize_canvas(width, height)` - aktualizacja wymiarów renderowania,
- `set_resource_base(path)` - konfiguracja ścieżki bazowej dla zasobów.

```
1  const changeModel = useCallback((modelName) => {
2    if (!moduleReady) {
3      console.warn('WebGPU module not ready yet.')
4      return false
5    }
6
7    if (typeof window.Module.change_model !== 'function') {
8      setError('WebAssembly module missing required function')
9      return false
10   }
11
12   try {
13     window.Module.change_model(modelName)
14     setCurrentModel(modelName)
15     return true
16   } catch (err) {
17     setError('Failed to change model')
18     return false
19   }
20 }, [moduleReady])
```

Listing 10.7: Wywołanie funkcji eksportowanej z C++

10.6.2 Przekazywanie elementu canvas

Element HTML canvas jest przekazywany do modułu WebAssembly przez właściwość `Module.canvas`. Emscripten automatycznie używa tego elementu do utworzenia kontekstu WebGPU:

```
1 moduleConfig.canvas = canvasRef.current
2
3 // Po inicjalizacji modul automatycznie używa przypisanego canvas
4 // do renderowania przez WebGPU
```

Listing 10.8: Przypisanie canvas do modułu

10.7 Komponent WebGPUCanvas

Komponent `WebGPUCanvas` stanowi publiczny interfejs do osadzania komponentu renderującego 3D w dowolnym miejscu aplikacji. Obsługuje responsywne wymiarowanie oraz automatyczną zmianę modelu.

```
1 <WebGPUCanvas
2   width={800}
3   height={600}
4   model="fourareen"
5   showControls={true}
6   onModelChange={
7     (model) => console.log('Model changed:', model)
8   }
9 />
```

Listing 10.9: Użycie komponentu `WebGPUCanvas`

Komponent obsługuje trzy tryby wymiarowania:

- określone wymiary (`width/height`) - stały rozmiar w pikselach,
- proporcjonalne - podanie tylko wysokości automatycznie oblicza szerokość (proporcja 4:3),
- responsywne - brak wymiarów powoduje dopasowanie do kontenera rodzica z użyciem `ResizeObserver`.